

The Same Word Twice

Class 09 Instructor Notes | 40 minutes of a 75-minute period (estimate — see Timing) | Four-page handout + skeleton notebook | Teams of 2–3

Placement and purpose

Follows Class 08 (matplotlib) and lab 5's introduction of comparison operators and boolean arrays. This is the first activity to treat conditional probability formally.

The one idea: a comparison applied to a column returns a boolean array, and averaging a boolean array is exactly conditioning. `np.mean(condition)` is $P(A)$; restrict the table first and it becomes $P(A|B)$. An `if` statement and the phrase “given that” are the same move — narrow down to a subset — spelled two different ways. That is the whole joke in the title, and Discussion D1 is where students are asked to say it in their own words.

History, for context. An earlier draft opened with a paper “Predicate Golf” warm-up (writing boolean rules to isolate marked patients) and closed with a third discussion question on overfitting to new-hospital data. Both were cut to fit a 40-minute slot — this class needed 20 minutes back for Quiz 1 review — and the remaining two parts were retimed and renumbered (dice is now Part 1, the notebook is now Part 2) rather than left with a gap where Part 1 used to be. Nothing below depends on either cut piece; both are recoverable from git history if a future session has room for them again.

Timing

This class has not been taught yet, so the timing below is a proposed split of the 75-minute period based on question count and code load, not a confirmed number from a live run. Adjust after the first time through and replace this block with real numbers.

10 min Part 1 — Thirty-Six Outcomes, the dice grid
25 min Part 2 — open `Class09_Same_Word_Twice_Skeleton.ipynb`, Q2.1–2.6
5 min Discussion D1–D2

If you are behind: Part 1 is the one to compress — it's the most self-contained of the two and the least essential once students have seen one worked conditioning example. Do not cut Discussion D2; it's the only place the class connects to something students will actually encounter outside this room (a test result and what “95% accurate” does and doesn't tell you). Part 2's 2.5 already asks for only “one quick pass” at a rule — don't let a team turn that into open-ended iteration.

Data file

`data/heart.csv` is in place and confirmed: 1,025 rows, columns `age`, `sex`, `cp`, `trestbps`, `chol`, `fbs`, `restecg`, `thalach`, `exang`, `oldpeak`, `slope`, `ca`, `thal`, `target` — matches the handout exactly. Every number in Part 2 below was produced by running the skeleton notebook's own code (`Table.read_table`, `.where`, `.apply`, the `score` function) against this file in `ds313`, not estimated.

Code blanks

Lighter than most classes — Part 2 has exactly one line with a fixed correct answer; the rest of the notebook is open-ended rule design with no single right code.

```
sick = heart.where('target', 1)
np.mean(sick.column('sex') == 0)
```

This is the Q2.2 blank. It mirrors the worked example just above it (`women = heart.where('sex', 0)` then `np.mean(women.column('target') == 1)`) with the condition and target reversed — same two columns, same numerator, different denominator. That reversal is the entire point of 2.2, so resist giving it away before students attempt it.

`my_rule(...)` and its `score(...)` call are intentionally open — any signature and any `if/elif/else` logic is fair game as long as the argument names passed to `apply` match the function’s parameters in order. The prompt directly above it now tells students to copy `example_rule` and change one thing rather than design from scratch — with no time built in for real exploratory data analysis, guessing thresholds cold against ten unfamiliar columns was not a realistic ask for a 25-minute block, and nudging a rule that already works is.

10 min — Part 1. Thirty-Six Outcomes

Standard equally-likely-outcomes grid; verified by direct count, not by running code.

- **1.1:** six cells sum to 7 (the full anti-diagonal) — $P(7) = 6/36 = 1/6$.
- **1.2:** every row contains exactly one 7, so each row’s count is $1/6$ — identical across all six rows. That constancy **is** independence: knowing the first die changes nothing about the chance of a 7.
- **1.3:** 6 sevens + 2 elevens (die pairs (5,6) and (6,5)) = 8 winning cells, $P(\text{win}) = 8/36 = 2/9 \approx 0.222$.
- **1.4:** row 3 has one winning cell (the 7 at (3,4)) → $P(\text{win} \mid \text{first} = 3) = 1/6 \approx 0.167$. Row 5 has two (7 at (5,2), 11 at (5,6)) → $P(\text{win} \mid \text{first} = 5) = 2/6 = 1/3 \approx 0.333$. Neither equals 1.3’s $2/9$, and the two rows disagree with each other — unlike 1.2, the first die now **does** change the answer, because “win” bundles two different diagonals together asymmetrically across rows.
- **1.5:** of the 8 winning cells, 2 have first die = 5 (the same two cells from 1.4’s row-5 count) → $P(\text{first} = 5 \mid \text{win}) = 2/8 = 1/4 = 0.25$. Compare with 1.4’s $P(\text{win} \mid \text{first} = 5) = 1/3$ — same two cells on top, a different denominator (8 winning cells vs. 6 cells in row 5) underneath. The one-sentence answer to “what is the denominator of a conditional probability”: the size of the subset you’ve restricted to — the condition, not the whole sample space.

25 min — Part 2. Write a Diagnostic Rule

2.1 $P(\text{disease})$ over all 1,025 patients: **526/1025 = 0.513** (51.3%). This is the unconditional baseline everything else in Part 2 gets compared against.

2.2 `women = heart.where('sex', 0)` gives **312 patients**; $P(\text{disease} \mid \text{female}) = 0.724$ (72.4%). `sick = heart.where('target', 1)` gives **526 patients**; $P(\text{female} \mid \text{disease}) = 0.430$ (43.0%). The grid-covering move from Part 1 is the same one here — 2.2’s first cell restricts by sex and asks about disease, the reverse restricts by disease and asks about sex. Same two columns, same numerator count, different denominator — structurally identical to 1.4 vs. 1.5.

Worth pointing out live: overall, women are $312/1025 = 30.4\%$ of the table, but 43.0% of the disease cases — women are **over-represented** among the sick relative to their share of the sample, and $P(\text{disease} \mid \text{female}) = 72.4\%$ sits well above the 51.3% baseline. That’s a large enough gap to be worth a “why might that be” aside — this is a referred-for-testing sample, not a general population, so it says something about who gets referred as much as it says anything about disease risk by sex. Don’t let the room read it as a general epidemiological fact.

2.3 (example_rule): flags **506 of 1,025** patients. $P(\text{flag} \mid \text{disease}) = 0.565$ (56.5%), $P(\text{disease} \mid \text{flag}) = 0.587$ (58.7%) — both only modestly above chance ($P(\text{disease}) = 51.3\%$), which matches the handout’s own description of this rule as “deliberately mediocre.”

Right after `score(calls)` runs on this rule, a new markdown cell — “What would a better rule look like?” — uses these exact two numbers to spell out the tradeoff: loosening a rule’s conditions tends to push

$P(\text{flag}|\text{disease})$ up and $P(\text{disease}|\text{flag})$ down, tightening tends to do the reverse, and a rule that gains on one axis while collapsing on the other (which `flag_everyone`, next, is built to demonstrate) is not actually better. This was added because a team can walk through 2.3/2.4 correctly and still not see why both numbers have to move together for a rule to genuinely improve — worth confirming out loud that they’ve internalized it before they start 2.5.

Standard names, for your own reference (not introduced to students): $P(\text{flag}|\text{disease})$ is **recall** (sensitivity, true positive rate) — $TP/(TP + FN)$ — and its complement is the false negative rate: the fraction of actually-sick patients the rule misses. $P(\text{disease}|\text{flag})$ is **precision** (positive predictive value) — $TP/(TP + FP)$ — and its complement is the false discovery rate: the fraction of flagged patients who are false alarms. The two denominators are different populations of the same 2×2 table — actually-sick vs. actually-flagged — which is why a rule can crush one number while doing nothing for the other: false negatives live in the column recall ignores, false positives live in the row precision ignores. This is the same pair as D2’s “sensitivity” and the (unnamed there) positive predictive value, just in a different setting — worth pointing out the parallel if a team makes the connection on their own, but neither term needs to be introduced for the activity to work.

2.4 (`flag_everyone`): flags **all 1,025**. $P(\text{flag}|\text{disease}) = 1.000$ exactly and $P(\text{disease}|\text{flag}) = 0.513$ exactly — matching $P(\text{disease})$ from 2.1 to three decimal places, which is not a coincidence: when every patient is flagged, “flagged” and “whole table” are the same set, so $P(\text{disease}|\text{flag})$ **is** $P(\text{disease})$ by construction, no matter what the data says. That’s the rule that’s perfect on one axis and useless on the other, and it’s the fact 2.4 is actually testing for.

2.5–2.6: no fixed answer — this is the open design space. The handout’s 2.5 says “one quick pass is fine” — don’t let a team iterate for the full 25 minutes; that’s what makes this fit in 40. Push teams to state up front **which** of the two numbers they’re optimizing before they tune thresholds (2.6 asks for exactly this), since a rule that’s “better” on one axis and worse on the other is the default outcome, not an edge case.

The notebook ends with a short markdown cell, “Where this goes next,” naming the nearest-neighbor method in Lab 10 and flagging the train/test distinction as a difference from today’s exercise. It’s there to close the loop, not to teach the method — no need to say anything more about it live unless a team asks.

5 min — Discussion

D1: the shared idea is restriction to a subset. An `if` restricts which branch of code runs, given that some condition holds; a conditional probability restricts the sample space to the subset where the condition holds, before computing a fraction. Push for the word “subset” to show up in the answer, not just “when” or “if” repeated back.

D2: “95% accurate” is almost always quoted as something like $P(\text{positive test} | \text{disease})$ (sensitivity) or overall accuracy across both classes — a statement about the test, conditioned on the true status. The patient wants $P(\text{disease} | \text{positive test})$ — a statement about them, conditioned on the test result. Getting from one to the other (Bayes’ theorem) requires the **prevalence** — how common the disease is in the population being tested — which is exactly the missing piece in “95% accurate” on its own. This is the same-word-twice idea landing on a case that matters: mixing up which probability is which is a real, common misreading of medical test results, not just a classroom exercise.

Notes on what was left out

No formal statement of Bayes’ theorem — D2 asks students to name the missing piece (prevalence) without deriving the formula that combines it. If a future class continues into formal Bayes’ theorem, this activity is the natural setup for it.

No discussion of overfitting or train/test evaluation. Part 2’s diagnostic rule is tuned by trial and error against the same 1,025 patients used to grade it — exactly the setup for that conversation — but the activity doesn’t

raise it. If a later class introduces train/test splits, it can call back to “the same 1,025 patients you used to grade it” from this activity.